

Общая структура ВКР выглядит так:

Введение (уже готово)

Глава 1. Теоретические основы мобильной разработки (фокус на базовых инструментах и технологиях, с обоснованием выбора IDE, языка и платформы).

Глава 2. Анализ альтернативных платформ и подходов к реализации системы управления мероприятиями (здесь разбор "почему не другие варианты", с аргументами на основе рынка и практических соображений).

Глава 3. Разработка мобильной части системы управления мероприятиями (практическая часть: доменный слой, взаимодействие с backend, UI, рекомендательная система).

Заключение (итоги, перспективы).

Библиографический список (ссылки на источники, включая использованные в аргументации).

Приложение (код, скриншоты, репозиторий).

Глава 1. Теоретические основы мобильной разработки

Эта глава закладывает фундамент, описывая ключевые технологии для твоего проекта. Здесь интегрируем аргументы за IntelliJ IDEA и кратко за Kotlin (подробно — во второй главе).

1.1. Обзор платформы Android и языка программирования Kotlin. Описание эволюции Android как ОС для смартфонов и планшетов: компоненты (активности, сервисы), преимущества (открытость, интеграция с Google-сервисами). Кратко о Kotlin: статическая типизация, корутины для асинхронных задач, совместимость с Java. Упомянуть, что Kotlin рекомендован Google для Android с 2017 года, что упрощает разработку приложений вроде системы событий.

1.2. Инструменты разработки: выбор среды IntelliJ IDEA. Сравнение IDE: Android Studio (специализирована для Android, с готовыми эмуляторами), Visual Studio Code (лёгкая, но требует плагинов), Eclipse (устаревшая). Аргументация за IntelliJ IDEA 2025.2.4: универсальность (основа для Android Studio, но гибче для мультязычных проектов), мощный анализ кода (снижает ошибки на 20-30% по сравнению с базовыми редакторами), встроенная поддержка Kotlin и Gradle. Минусы: ручная настройка Android-плагинов, но это окупается для проектов без сложного тестирования. Выбор обоснован экономией времени и отсутствием нужды в специализированном оборудовании.

1.3. Архитектурные принципы и паттерны проектирования в мобильной разработке. Обзор MVVM сравнение с MVP. Роль Hilt для внедрения зависимостей: упрощает модульность.

1.4. Методы создания пользовательского интерфейса в Android-приложениях. Переход от XML к Jetpack Compose: declarative подход для экранов (примеры из твоего кода, как Composable для карточек событий). Преимущества: реактивность, Material Design.

1.5. Организация работы с данными и сетевым взаимодействием. Инструменты: DataStore для аутентификации, Flow для потоков данных, репозитории для backend. Обоснование: обеспечивает безопасность и эффективность в клиент-серверных системах.

Глава 2. Анализ альтернативных платформ и подходов к реализации системы управления мероприятиями

Здесь фокус на сравнениях и аргументах "почему не иначе". Это позволит не комкать анализ в первой главе.

2.1. Сравнение языков программирования для мобильной разработки: выбор Kotlin. Сравнение с Java (основной альтернативой): Kotlin короче на 20-40% (меньше boilerplate-кода), имеет null-safety (снижает краши на 60%), корутины вместо потоков для асинхронности. С C# или Swift: не подходят для Android без кросс-платформенных фреймворков. Выбор Kotlin: официальная поддержка Google, лёгкий переход от Java, подходит для твоего проекта с асинхронной загрузкой событий.

2.2. Выбор платформы: Android в сравнении с iOS. Доля рынка: Android — 72% глобально в 2026 году (доминирует в развивающихся странах), iOS — 26-28% (преимущественно в США и премиум-сегменте). Аргументы за Android: открытость (проще внедрение и кастомизация), отсутствие нужды в Apple-устройствах для тестирования (экономия ресурсов), ниже стоимость разработки. Против iOS: закрытая система (сложно внедрять корпоративные приложения без App Store), ограничения на тестирование (нужен Mac и iPhone), что не подходит для твоего проекта без специального оборудования.

2.3. Сравнение форматов реализации: мобильное приложение vs веб-сайт. Преимущества мобильного: оффлайн-доступ (просмотр событий без интернета), push-уведомления (напоминания о мероприятиях), интеграция с устройством (геолокация для оффлайн-событий, календарь). Веб-сайт: требует браузера, медленнее на мобильных, нет оффлайн-режима. Выбор мобильного: лучше для повседневного использования в организациях, где пользователи часто в движении.

2.4. Сравнение с чат-ботами в мессенджерах. Чат-боты: дешевы в разработке, доступны в Telegram/Viber, но ограничены текстовым интерфейсом, зависят от платформы (риски блокировок), нет оффлайн-доступа. Мобильное приложение: полный UI (карточки событий, пагинация), персонализация (рекомендации),

независимость. Выбор: чат-боты подходят для простых запросов, но не для комплексной системы управления мероприятиями с анализом данных.

Глава 3. Разработка мобильной части системы управления мероприятиями (5 подразделов, объём 15-20 страниц)

Здесь практическая часть, основанная на твоём коде.

3.1. Разработка доменного слоя (модели Event, User, use-case'ы).

3.2. Взаимодействие с backend-частью (репозитории, корутины).

3.3. Проектирование пользовательского интерфейса (экраны, Compose).

3.4. Интеграция рекомендательной системы (на основе предпочтений пользователей).

3.5. Тестирование и оптимизация приложения.